

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
КАФЕДРА АВТОМАТИЗАЦІЇ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

ЗВІТ

ЗВІТ З ЛАБОРАТОРНИХ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
«МЕТОДИ ТА СИСТЕМИ ОБЧИСЛЮВАЛЬНОГО ІНТЕЛЕКТУ»

Виконав студент групи КН-23-1

Полинько Ігор Миколайович

Перевірів: старший викладач кафедри ОМНІ Юдіна А. Л.

Кременчук 2026

ЛАБОРАТОРНА РОБОТА № 6

Тема: Алгоритмічна реалізація процедури фаззифікації

Мета: закріплення теоретичних знань і придбання навичок програмної реалізації алгоритму фаззифікації

Порядок виконання роботи:

Розробити алгоритм, у якому зафіксована кількість вхідних шкал і кількість термів. В алгоритмі реалізувати такі функції.

- уводити діапазон зміни значень параметрів на кожній вхідній шкалі (наприклад, від 0 до 10);
- уводити значення точок на кожній шкалі, що відображає параметри функцій приналежності (ядро, носій, межі і т.д.);
- уводити значення вхідних змінних за кожною шкалою;
- обчислювати і зберігати в масивах значення функцій приналежності для кожного терму кожної універсальної множини (табл. 6.2);
- виводити вміст указаних масивів з указівкою назви терму і точки на шкалі універсальної множини.

Таблиця 6.1 – Варіанти завдань

№ варіанта	Універсальна множина	Терм 1	Терм 2	Терм 3
15	A	$a_1 = 10$ $b_1 = 0,5$	$a_2 = 20$ $b_2 = 0,4$	$a_3 = 30$ $b_3 = 0,5$
	B	$c_1 = 160$ $b_1 = 175$	$a_2 = 154$ $b_2 = 210$ $c_2 = 190$	$a_3 = 200$ $c_3 = 215$
	C	$a_1 = 0,1$ $b_1 = 1$ $c_1 = 40$	$a_2 = 75$ $b_2 = 134$ $c_2 = 90$ $d_2 = 110$	$a_3 = 0,09$ $b_3 = 1,5$ $c_3 = 150$

Варіант 15, Полинсько

Таблиця 6.2 – Формули для використання

Універсальна множина	Терм 1	Терм 2	Терм 3
A	$\mu_{a1} = \begin{cases} 1, x < a_1 \\ \exp\left[-\left \frac{x - a_1}{2b_1}\right \right], \\ x < a_1 \end{cases}$	$\mu_{a2} = \exp\left[-\left \frac{x - a_2}{2b_2}\right \right]$	$\mu_{a3} = \begin{cases} 1, x > a_3 \\ \exp\left[-\left \frac{x - a_3}{2b_3}\right \right], \\ x > a_3 \end{cases}$
B	$\mu_{b1} = \begin{cases} 1, x < c_1 \\ \frac{b_1 - x}{b_1 - c_1}, c_1 < x < b \\ 0, x \geq b \end{cases}$	$\mu_{b2} = \begin{cases} 0, x \leq a_2 \\ \frac{x - a_2}{b_2 - c_2} \\ c_2 < x < b_2 \\ 0, x \geq b_2 \end{cases}$	$\mu_{b3} = \begin{cases} 0, x \leq a_3 \\ \frac{x - a_3}{c_3 - a_3} \\ a_3 < x < c_3 \\ 1, x \geq c_3 \end{cases}$
C	$\mu_{c1} = \begin{cases} 1, x < c_1 \\ \{1 + [a_1(x - c_1)]^{b_1}\}^{-1} \\ x > c_1 \end{cases}$	$\mu_{c2} = \begin{cases} 0, x \leq a_2 \\ \frac{x - a_2}{c_2 - a_2} \\ a_2 < x < c_2 \\ 1, c_2 \leq x \leq d_2 \\ \frac{b_2 - x}{b_2 - d_2} \\ d_2 < x < b_2 \\ 0, x \geq b_2 \end{cases}$	$\mu_{c3} = \begin{cases} 1, x \geq c_3 \\ \{1 + [a_3(c_3 - x)]^{b_3}\}^{-1} \\ x < c_3 \end{cases}$

Скрипт програми:

```
import numpy as np

# =====
# НЕЧІТКА СИСТЕМА (Варіант 15, Полинсько)
# Повний цикл:
# 1. Фазифікація
# 2. База правил
# 3. Нечіткий висновок
# 4. Агрегація
# 5. Дефазифікація
# =====

# =====
# 1. НАЗВИ ТЕРМІВ
# =====

terms = ["LOW", "MEDIUM", "HIGH"]

# =====
# 2. ПАРАМЕТРИ ФУНКЦІЙ ПРИНАЛЕЖНОСТІ
# =====
```

```

# ----- A -----
a_A = [10, 20, 30]
b_A = [0.5, 0.4, 0.5]

# ----- B -----
c_B = [160, None, None]
b_B = [175, 210, None]
a_B = [None, 154, 200]

c2 = 190
c3 = 215

# ----- C -----
a_C = [0.1, 75, 0.09]
b_C = [1, 134, 1.5]
c_C = [40, 90, 150]
d_C = [None, 110, None]

# =====
# 3. ФУНКЦІЇ ПРИНАЛЕЖНОСТІ
# =====

# ----- A -----

def mu_A1(x):
    if x < a_A[0]:
        return 1
    else:
        return np.exp(-abs((x - a_A[0]) / (2 * b_A[0])))

def mu_A2(x):
    return np.exp(-abs((x - a_A[1]) / (2 * b_A[1])))

def mu_A3(x):
    if x > a_A[2]:
        return 1
    else:
        return np.exp(-abs((x - a_A[2]) / (2 * b_A[2])))

# ----- B -----

def mu_B1(x):
    if x < c_B[0]:
        return 1
    elif c_B[0] < x < b_B[0]:
        return (b_B[0] - x) / (b_B[0] - c_B[0])
    else:
        return 0

def mu_B2(x):
    if x <= a_B[1]:
        return 0
    elif a_B[1] < x < c2:
        return (x - a_B[1]) / (c2 - a_B[1])
    elif c2 <= x < b_B[1]:
        return 1
    else:
        return 0

def mu_B3(x):
    if x <= a_B[2]:
        return 0
    elif a_B[2] < x < c3:
        return (x - a_B[2]) / (c3 - a_B[2])

```

```

        else:
            return 1

# ----- C -----

def mu_C1(x):
    if x < c_C[0]:
        return 1
    else:
        return 1 / (1 + (a_C[0] * (x - c_C[0])) ** b_C[0])

def mu_C2(x):
    if x <= a_C[1]:
        return 0
    elif a_C[1] < x < c_C[1]:
        return (x - a_C[1]) / (c_C[1] - a_C[1])
    elif c_C[1] <= x <= d_C[1]:
        return 1
    elif d_C[1] < x < b_C[1]:
        return (b_C[1] - x) / (b_C[1] - d_C[1])
    else:
        return 0

def mu_C3(x):
    if x >= c_C[2]:
        return 1
    else:
        return 1 / (1 + (a_C[2] * (c_C[2] - x)) ** b_C[2])

# =====
# 4. ВВІД ДАНИХ
# =====

x_A = 15
x_B = 205
x_C = 90

# =====
# 5. ФАЗИФІКАЦІЯ
# =====

AF_A = [
    mu_A1(x_A),
    mu_A2(x_A),
    mu_A3(x_A)
]

AF_B = [
    mu_B1(x_B),
    mu_B2(x_B),
    mu_B3(x_B)
]

AF_C = [
    mu_C1(x_C),
    mu_C2(x_C),
    mu_C3(x_C)
]

# =====
# 6. ВИВЕДЕННЯ РЕЗУЛЬТАТІВ ФАЗИФІКАЦІЇ
# =====

print("\n=====")
print("РЕЗУЛЬТАТИ ФАЗИФІКАЦІЇ")

```

```

print("=====")

print("\nМножина A:")
for i in range(3):
    print(f"{terms[i]} = {AF_A[i]:.4f}")

print("\nМножина B:")
for i in range(3):
    print(f"{terms[i]} = {AF_B[i]:.4f}")

print("\nМножина C:")
for i in range(3):
    print(f"{terms[i]} = {AF_C[i]:.4f}")

# =====
# 7. БАЗА ПРАВИЛ
# =====

# HIGH = индекс 2
A_high = AF_A[2]
B_high = AF_B[2]
C_high = AF_C[2]

# =====
# 8. НЕЧІТКИЙ ВИСНОВОК
# =====

# ----- ПРАВИЛО 1 -----
# IF A high AND B high AND C high
# THEN Risk = high

risk_high = min(A_high, B_high, C_high)

# ----- ПРАВИЛО 2 -----
# IF A high OR B high OR C high
# THEN Risk = medium

risk_medium = max(A_high, B_high, C_high)

# ----- ПРАВИЛО 3 -----
# IF A NOT high AND B NOT high AND C NOT high
# THEN Risk = low

risk_low = min(
    1 - A_high,
    1 - B_high,
    1 - C_high
)

# =====
# 9. АГРЕГАЦІЯ
# =====

print("\n=====")
print("РЕЗУЛЬТАТИ ПРАВИЛ")
print("=====")

print(f"Risk LOW      = {risk_low:.4f}")
print(f"Risk MEDIUM    = {risk_medium:.4f}")
print(f"Risk HIGH       = {risk_high:.4f}")

# =====
# 10. ДЕФАЗИФІКАЦІЯ
# =====

```

```

# Центри термів
centers = {
    "low": 20,
    "medium": 50,
    "high": 80
}

# Формула центру ваги
numerator = (
    risk_low * centers["low"] +
    risk_medium * centers["medium"] +
    risk_high * centers["high"]
)

denominator = (
    risk_low +
    risk_medium +
    risk_high
)

# Захист від ділення на нуль
if denominator == 0:
    centroid = 0
else:
    centroid = numerator / denominator

# =====
# 11. ФІНАЛЬНИЙ РЕЗУЛЬТАТ
# =====

print("\n=====")
print("ДЕФАЗИФІКАЦІЯ")
print("=====")

print(f"Centroid = {centroid:.2f}")

print(f"\nСтупінь ризику = {centroid:.2f}%")

```

Результат:

```
=====
РЕЗУЛЬТАТИ ФАЗИФІКАЦІЇ
=====

Множина А:
LOW = 0.0067
MEDIUM = 0.0019
HIGH = 0.0000

Множина В:
LOW = 0.0000
MEDIUM = 0.0000
HIGH = 0.6667

Множина С:
LOW = 0.1667
MEDIUM = 1.0000
HIGH = 0.0738

=====
РЕЗУЛЬТАТИ ПРАВИЛ
=====

Risk LOW    = 0.3333
Risk MEDIUM = 0.6667
Risk HIGH    = 0.0000

=====
ДЕФАЗИФІКАЦІЯ
=====

Centroid = 40.00

Ступінь ризику = 40.00%
```

Рисунок 6.1 – Результат роботи програми

Висновок: на цій лабораторній роботі ми закріпили теоретичні знання і придбали навички програмної реалізації алгоритму фаззифікації

Контрольні питання:

1. Пояснити роль і місце процедури фаззифікації в алгоритмі нечіткого логічного виведення

Фаззифікація є першим етапом нечіткого логічного виведення. Її задача – перетворити чіткі (звичайні числові) входні дані у нечіткі значення. Тобто замість конкретного числа система отримує ступені належності до різних термів (наприклад: «малий», «середній», «великий»).

2. Назвіть усі основні етапи алгоритму нечіткого логічного виведення

Основні етапи такі:

- фаззифікація (перетворення входних даних у нечіткі значення)
- застосування правил (база знань, типу «якщо-то»)
- агрегування результатів правил
- дефаззифікація (перетворення результату назад у чітке число)

3. Пояснити роль функцій приналежності

Функції приналежності показують, наскільки конкретне значення належить до певного терму.

Їх значення лежать у діапазоні від 0 до 1.

Наприклад, якщо температура 25:

- «низька» $\rightarrow 0.2$
- «середня» $\rightarrow 0.7$
- «висока» $\rightarrow 0.1$

Тобто функції приналежності дозволяють описати нечіткість і плавні переходи між станами.

4. Пояснити, як результати фаззифікації застосовуються у алгоритмі логічного виведення

Результати фаззифікації використовуються як вхід для правил нечіткої логіки. Кожне правило оцінюється з урахуванням ступенів належності.

Наприклад:

«Якщо температура висока $\rightarrow$ вентилятор швидкий»

Якщо «висока» = 0.7, то і правило спрацює з силою 0.7.

Далі ці значення комбінуються, і на їх основі формується підсумковий результат.